

CAHIER DES CHARGES

RESHAPE

Plateforme SaaS de Migration de Données

Remodeler, reformater, transformer vos données

VERSION	DATE	STATUT	CLASSIFICATION
1.0	Mars 2026	MVP — Document initial	Confidentiel

TABLE DES MATIÈRES

Placeholder for table of contents	0
-----------------------------------	---

1. Introduction et contexte

1.1. Présentation du projet

Ce document constitue le cahier des charges fonctionnel et technique de **Reshape**, une plateforme SaaS de migration de données inter-applicatives. Le produit vise à simplifier et automatiser le transfert de données entre deux systèmes logiciels du même secteur ou de secteurs différents, en proposant une expérience guidée, sécurisée et tarifiée par crédits.

Cas d'usage typique : une entreprise du BTP utilise un logiciel métier devenu obsolète et souhaite migrer l'ensemble de ses données (clients, projets, factures, etc.) vers un nouveau logiciel. Reshape reçoit en entrée l'ancienne modélisation de données et le format cible, puis produit en sortie les données transformées, prêtes à être importées.

1.2. Objectifs stratégiques

- Permettre à toute entreprise de migrer ses données entre deux systèmes sans compétence technique approfondie
- Offrir aux intégrateurs et consultants IT un outil professionnel pour réaliser des migrations pour le compte de leurs clients
- Proposer un modèle économique flexible basé sur la consommation réelle (crédits)
- Garantir la sécurité et la confidentialité des données migrées (chiffrement, isolation, RGPD)
- S'imposer comme la référence SaaS généraliste pour la migration de données inter-applicatives

1.3. Cibles utilisateurs

La plateforme s'adresse à deux segments principaux, servis simultanément dès le lancement :

- **TPE/PME** qui changent de logiciel métier et doivent transférer leurs données historiques
- **Intégrateurs, consultants IT et ESN** qui réalisent des migrations pour le compte de leurs clients

■ L'approche est **généraliste** : aucune restriction de secteur. La plateforme fonctionne pour le BTP, l'assurance, la santé, le retail, la logistique ou tout autre domaine.

1.4. Périmètre du MVP

Ce cahier des charges définit le périmètre du MVP (Minimum Viable Product). Les fonctionnalités marquées « post-MVP » sont identifiées mais ne font pas partie de la première livraison.

2. Sources et destinations de données

2.1. Formats de fichiers supportés

Les formats suivants sont acceptés en entrée (source) et en sortie (destination) :

Format	Extensions	Détails
CSV / TSV	.csv, .tsv	Détection automatique du délimiteur et de l'encodage (UTF-8, Latin-1, etc.)
Excel	.xlsx, .xls	Support multi-feuilles, détection des en-têtes
Archive ZIP	.zip	Contenant un ensemble de fichiers CSV et/ou Excel, décompressés et analysés individuellement
Dump SQL	.sql	Parsing des CREATE TABLE (extraction du schéma) et des INSERT INTO (extraction des données)

2.2. Connexions directes aux bases de données

La plateforme permet une connexion directe aux bases de données suivantes, en source comme en destination :

SGBD	Protocole	Authentification
PostgreSQL	TCP/IP (port 5432)	Utilisateur + mot de passe + nom de la base
MySQL / MariaDB	TCP/IP (port 3306)	Utilisateur + mot de passe + nom de la base
SQL Server	TCP/IP (port 1433)	Utilisateur + mot de passe + instance
MongoDB	TCP/IP (port 27017)	URI de connexion ou user/password + base

Les credentials de connexion sont chiffrées en AES-256 côté applicatif avec une clé maître (voir section 8 — Sécurité).

2.3. Format de sortie

L'utilisateur choisit le mode de livraison des données transformées :

- **Fichier téléchargeable** : les données sont exportées dans le format choisi (CSV, Excel, SQL) et mises à disposition en téléchargement

- **Push direct** : les données sont injectées directement dans la base de données cible via la connexion fournie par l'utilisateur

Les deux modes peuvent coexister dans une même migration (ex : push dans la DB + export CSV de contrôle).

2.4. Gestion des volumes

Il n'y a pas de limite technique sur la taille des fichiers ou le nombre de lignes. Le volume est uniquement contrôlé par le solde de crédits de l'utilisateur. Les migrations volumineuses sont traitées de manière asynchrone via une file de messages (RabbitMQ).

3. Mapping et transformation des données

3.1. Analyse automatique du schéma

Dès l'upload d'une source (fichier ou connexion DB), la plateforme analyse automatiquement :

- La structure des tables/fichiers : noms de colonnes, types de données détectés
- Les relations entre tables : détection automatique des clés étrangères (FK) par analyse des noms, types et valeurs
- Les contraintes : clés primaires, unicité, nullabilité

■ L'utilisateur peut modifier, corriger ou compléter les relations détectées manuellement. Cette étape est critique car la détection automatique peut produire des faux positifs.

3.2. Modes de mapping

Deux modes de mapping sont proposés, au choix de l'utilisateur :

3.2.1. Mapping manuel

L'utilisateur définit lui-même les correspondances champ par champ entre le schéma source et le schéma cible. Interface drag-and-drop ou formulaire de sélection.

3.2.2. Mapping assisté par IA

Un moteur d'intelligence artificielle analyse les noms de colonnes, les types et un échantillon de données pour proposer un mapping automatique. L'utilisateur valide, ajuste ou rejette chaque suggestion. Ce mode entraîne un **surcoût de x1.2** sur la consommation de crédits.

3.3. Transformations supportées

Au-delà du simple renommage de colonnes, les transformations suivantes sont disponibles au MVP :

Transformation	Description	Exemple
Conversion de types	Changement de format de dates, nombres, booléens	dd/MM/yyyy → yyyy-MM-dd
Concaténation	Fusion de plusieurs champs en un seul	prénom + nom → nom_complet

Transformation	Description	Exemple
Split	Découpage d'un champ en plusieurs	adresse → rue, ville, CP
Nettoyage	Trim, changement de casse, suppression d'accents	" PARIS " → "Paris"
Valeurs par défaut	Remplacement des champs vides par une valeur configurable	null → "Non renseigné"
Formules custom	Expressions personnalisées pour des transformations complexes	prix_ht * 1.20 → prix_ttc

3.4. Préservation des relations

Lorsque la source contient des clés étrangères, la plateforme :

1. Détecte automatiquement les relations (FK) entre tables
2. Affiche un graphe visuel des dépendances pour validation
3. Permet à l'utilisateur de corriger les relations détectées
4. Préserve l'intégrité référentielle lors de la transformation (ré-indexation des ID si nécessaire)

3.5. Preview obligatoire (Dry-run)

Avant toute exécution de migration, un mode preview est obligatoire :

- Affichage d'un échantillon des données transformées (10 à 50 premières lignes par table)
- Rapport de validation : erreurs détectées, avertissements, statistiques
- L'utilisateur doit explicitement confirmer avant de lancer l'exécution réelle
- **Les 3 premiers previews sont gratuits**, les suivants consomment des crédits (coût réduit)

3.6. Gestion des erreurs

L'utilisateur configure le comportement en cas d'erreur sur des lignes individuelles :

- **Stopper** : toute la migration est annulée à la première erreur (rollback)
- **Ignorer et reporter** : les lignes en erreur sont exclues et exportées dans un fichier d'erreurs séparé avec le détail de chaque erreur

3.7. Templates de mapping

Un mapping validé peut être sauvegardé comme template réutilisable. Cela permet aux intégrateurs qui réalisent la même migration pour plusieurs clients de ne configurer le mapping qu'une seule fois.

- Nom et description du template
- Sauvegarde du mapping complet (correspondances, transformations, règles de validation)
- Réutilisation en un clic pour une nouvelle migration
- Modification possible sans impacter les migrations précédentes

4. Système de tarification par crédits

4.1. Unité de crédit

Le coût d'une migration est calculé selon un **mix pondéré** de trois facteurs :

- **Nombre de lignes** transformées
- **Complexité** des transformations appliquées (conversion simple, formule custom, etc.)
- **Volume** de données traité (Mo)

Une formule de calcul interne pondère ces trois paramètres pour produire un coût en crédits. Le détail de la formule est paramétrable côté administration.

4.2. Multiplicateur IA

Lorsque l'utilisateur opte pour le mapping assisté par IA, un **multiplicateur de x1.2** est appliqué au coût total en crédits de la migration. Le mapping manuel n'entraîne aucun surcoût.

4.3. Modèle tarifaire

Le modèle combine un essai gratuit limité et un système de pay-as-you-go :

Élément	Détail
Essai gratuit	Quota de crédits offerts à l'inscription, valable uniquement le premier mois. Pas de renouvellement.
Pay-as-you-go	Achat de packs de crédits à la demande. Les crédits achetés n'expirent pas.
Estimation préalable	Avant chaque migration, le coût estimé en crédits est affiché. L'utilisateur confirme ou annule.
Preview gratuit	Les 3 premiers dry-runs sont gratuits. Au-delà, un coût réduit est appliqué.

4.4. Passerelle de paiement

L'achat de crédits s'effectue via **Paddle**, qui gère la facturation, la TVA internationale et les différents moyens de paiement (carte bancaire, PayPal, etc.). Paddle agit comme Merchant of Record.

5. Authentification et gestion multi-tenant

5.1. Inscription et connexion

Deux modes d'authentification sont supportés :

- **Email + mot de passe** : inscription classique avec vérification par email
- **OAuth 2.0** : connexion via Google et/ou GitHub

L'authentification repose sur des tokens JWT avec refresh token.

5.2. Architecture multi-tenant

La plateforme adopte une architecture multi-tenant avec deux niveaux d'organisation :

5.2.1. Organisation (tenant)

- Chaque organisation est un tenant isolé logiquement (**tenant_id** sur toutes les tables)
- Une organisation possède son propre solde de crédits, ses templates, son historique
- Rôles : **Admin** (gestion des membres, achat de crédits, audit trail) et **Opérateur** (création et exécution de migrations)

5.2.2. Utilisateur simple

- Un utilisateur sans organisation fonctionne en mode individuel
- Il dispose de son propre solde de crédits et de son historique
- Il peut être invité à rejoindre une organisation à tout moment

5.3. Isolation des données

L'isolation est assurée par une séparation logique : chaque enregistrement porte un `tenant_id`. Toutes les requêtes sont automatiquement filtrées par le tenant courant au niveau applicatif (filtre global / intercepteur Spring).

5.4. Audit trail

Chaque action significative est enregistrée dans un journal d'audit accessible aux admins d'organisation :

- Qui a lancé quelle migration, quand, combien de crédits consommés
- Achats de crédits, ajout/suppression de membres

- Modifications de templates
- Connexions et tentatives de connexion

6. Architecture technique

6.1. Stack technologique

Composant	Technologie	Justification
Backend	Spring Boot (Java/Kotlin)	Robustesse, écosystème mature, expertise de l'équipe
Frontend	React / Next.js	Écosystème riche, SSR pour la landing page, flexibilité
Base de données	PostgreSQL	Fiabilité, support JSON, RLS natif
Stockage fichiers	MinIO (compatible S3)	Object storage auto-hébergé, performant, gratuit
File de messages	RabbitMQ	Jobs asynchrones fiables, dead letter queues, routing flexible
Paieement	Paddle	Merchant of Record, gestion TVA internationale
CI/CD	GitHub Actions	Intégration native avec le repo, pipelines déclaratifs

6.2. Architecture applicative

L'architecture retenue est un **monolithe modulaire**, organisé en modules fonctionnels faiblement couplés :

Module	Responsabilité
auth	Inscription, connexion, gestion des tokens JWT, OAuth
tenant	Gestion des organisations, membres, rôles, invitations
connector	Abstraction des sources/destinations (fichiers, DB), parsing, détection de schéma
mapper	Mapping manuel et assisté par IA, templates, gestion des relations FK
transformer	Moteur de transformation : conversions, formules, nettoyage, split/concat
migration	Orchestration du pipeline complet : analyse → mapping → preview → exécution
billing	Crédits, estimation de coût, intégration Paddle, historique d'achats
notification	Emails transactionnels, notifications in-app, webhooks
audit	Journal d'audit, logs d'actions

Module	Responsabilité
storage	Abstraction MinIO/S3, gestion du cycle de vie des fichiers (rétention 30j)

Cette architecture permet une évolution future vers des microservices si la charge le justifie, sans refonte majeure.

6.3. Pipeline de migration

Le pipeline de migration suit un flux linéaire orchestré par le module migration :

1. Upload / Connexion : l'utilisateur fournit la source (fichier ou credentials DB)
2. Analyse : parsing automatique du schéma, détection des FK, profiling des données
3. Définition de la cible : upload du schéma cible ou connexion à la DB cible
4. Mapping : correspondances source → cible (manuel ou IA)
5. Configuration des transformations : règles appliquées champ par champ
6. Estimation : calcul du coût en crédits, affichage à l'utilisateur
7. Preview (dry-run) : échantillon transformé, rapport de validation
8. Confirmation : l'utilisateur valide et lance l'exécution
9. Exécution asynchrone : traitement via RabbitMQ, progression en temps réel
10. Livraison : fichier téléchargeable et/ou push dans la DB cible
11. Notification : email + in-app + webhook

6.4. Traitement asynchrone

Les migrations sont exécutées de manière asynchrone via RabbitMQ :

- Un message est publié dans la queue pour chaque migration confirmée
- Un ou plusieurs workers consomment les messages et exécutent les transformations
- La progression est remontée en temps réel via WebSocket/SSE
- En cas d'échec, le message est redirigé vers une dead letter queue pour analyse

7. Hébergement et déploiement

7.1. Infrastructure

Composant	Hébergement	Détail
Backend Spring Boot	Contabo VPS + Coolify	Conteneur Docker, déploiement automatisé
Frontend Next.js	Vercel	SSR, CDN mondial, déploiement auto sur push
PostgreSQL	Contabo VPS + Coolify	Instance Docker, backups automatisés
MinIO	Contabo VPS + Coolify	Object storage, compatible S3
RabbitMQ	Contabo VPS + Coolify	Instance Docker dédiée

7.2. CI/CD

Le pipeline CI/CD repose sur GitHub Actions :

- Build et tests automatisés sur chaque push / pull request
- Analyse statique du code (linting, sécurité)
- Build de l'image Docker et push vers le registry
- Déploiement automatique via Coolify (backend) et Vercel (frontend)
- Environnements : development, staging, production

7.3. Monitoring

Au MVP, le monitoring reste basique :

- Logs Docker centralisés via Coolify
- Health checks sur les services critiques (backend, PostgreSQL, RabbitMQ, MinIO)
- Alertes en cas de service down (Coolify built-in)

L'évolution post-MVP prévoit l'ajout de Loki/Grafana ou ELK pour les logs, et des alertes Slack/Discord.

8. Sécurité et conformité

8.1. Chiffrement

- **En transit** : TLS 1.2+ obligatoire sur toutes les communications (HTTPS, connexions DB)
- **Au repos** : chiffrement AES-256 des fichiers stockés sur MinIO et des données sensibles en base
- **Credentials DB utilisateurs** : chiffrement applicatif AES-256 avec clé maître stockée séparément du code et de la base

8.2. Isolation multi-tenant

Séparation logique par tenant_id sur toutes les tables. Un filtre applicatif global (intercepteur Spring) garantit qu'aucune requête ne peut accéder aux données d'un autre tenant. Tests automatisés de non-fuite inter-tenant.

8.3. Rate limiting

Un rate limiting global est appliqué sur toutes les routes API pour prévenir les abus :

- Limite par IP et par utilisateur authentifié
- Limites renforcées sur les endpoints sensibles (auth, paiement)
- Réponse HTTP 429 avec header Retry-After

8.4. Conformité RGPD

Niveau basique au MVP :

- Suppression automatique des fichiers et résultats de migration après 30 jours
- Mentions légales, politique de confidentialité et CGU
- Droit à la suppression du compte et des données associées sur demande
- Pas de transfert de données hors UE (hébergement Contabo en Europe)

8.5. Sécurité applicative

- Protection CSRF, XSS, injection SQL (Spring Security + paramétrage strict)
- Validation des entrées côté serveur (taille de fichier, types MIME, formats)
- Dépendances scannées régulièrement (Dependabot / Snyk)
- Secrets gérés via variables d'environnement, jamais en dur dans le code

9. Expérience utilisateur et interface

9.1. Principes de design

- Interface sobre, professionnelle, orientée productivité
- **Dark mode** supporté dès le MVP (toggle dans les paramètres)
- **Internationalisation** : français et anglais dès le lancement
- Responsive design (desktop prioritaire, mobile consultable)

9.2. Pages principales

Page	Description
Landing page	Présentation du produit, fonctionnalités, tarifs, témoignages, CTA inscription. Intégrée au projet Next.js (SSR).
Dashboard	Vue d'ensemble : migrations en cours/terminées, solde de crédits, dernières activités, raccourcis.
Nouvelle migration	Flow step-by-step guidé : Source → Cible → Mapping → Transformations → Preview → Confirmation → Exécution.
Historique	Liste des migrations passées avec statut, date, crédits consommés, re-téléchargement possible.
Templates	Gestion des templates de mapping sauvegardés.
Crédits	Solde actuel, historique des consommations et achats, bouton d'achat.
Organisation	Gestion des membres (admin), rôles, invitations, audit trail.
Paramètres	Profil, sécurité, préférences (langue, thème), credentials DB sauvegardées.

9.3. Wizard de migration

Le flow de création d'une migration est un wizard linéaire en étapes. L'utilisateur ne peut passer à l'étape suivante qu'après validation de l'étape en cours. Chaque étape affiche une barre de progression et permet de revenir en arrière.

- 1. Source** : choix du type (fichier ou connexion DB), upload ou saisie des credentials
- 2. Analyse** : affichage du schéma détecté, relations FK, validation utilisateur
- 3. Cible** : définition du schéma cible (upload, connexion DB, ou saisie manuelle)

- 4. **Mapping** : correspondances source → cible (mode manuel ou IA)
- 5. **Transformations** : configuration des règles par champ
- 6. **Preview** : dry-run, échantillon, rapport de validation
- 7. **Estimation et confirmation** : coût en crédits, choix du mode de livraison, lancement

9.4. Notifications

Trois canaux de notification sont supportés :

- **Email** : notification à la fin d'une migration (succès ou échec), avec lien de téléchargement
- **In-app** : notifications en temps réel dans l'interface (badge, toast, panneau de notifications)
- **Webhook** : URL configurable, appelée avec un payload JSON à la fin de chaque migration (idéal pour les intégrateurs)

10. Historique et rétention des données

10.1. Historique des migrations

Chaque migration est archivée avec les informations suivantes :

- Identifiant unique, date de création, date de fin
- Statut : en attente, en cours, terminée, échouée
- Source et destination (type, nom du fichier ou connexion)
- Nombre de lignes traitées, nombre d'erreurs
- Crédits consommés
- Fichier de résultat téléchargeable (tant que la rétention est active)
- Fichier d'erreurs (si applicable)

10.2. Politique de rétention

- Les fichiers uploadés et résultats sont conservés **30 jours** après la fin de la migration
- Passé ce délai, les fichiers sont supprimés automatiquement de MinIO
- Les métadonnées (statut, dates, statistiques) sont conservées indéfiniment dans PostgreSQL
- L'utilisateur est prévenu par email **7 jours** avant la suppression

11. Équipe et organisation du projet

11.1. Composition de l'équipe

Le projet est porté par une petite équipe de 2 à 3 développeurs :

- **Lead dev / Architecte** : architecture backend, pipeline de migration, intégrations DB
- **Développeur frontend** : interface Next.js, wizard, dashboard, landing page
- **Développeur fullstack (optionnel)** : module IA, transformations, tests

11.2. Planning

Aucune deadline fixe n'est imposée. Le développement suit une approche itérative avec des sprints de 2 semaines. Jalons indicatifs :

Phase	Périmètre	Durée estimée
1 — Fondations	Auth, multi-tenant, CI/CD, structure monolithe, landing page	4-6 semaines
2 — Connecteurs	Upload fichiers, connexions DB, parsing, détection de schéma	4-6 semaines
3 — Mapping & Transfo	Mapping manuel, transformations, preview, gestion erreurs	4-6 semaines
4 — Exécution	Pipeline complet, RabbitMQ, workers, livraison	3-4 semaines
5 — Billing	Intégration Paddle, crédits, estimation, historique d'achats	2-3 semaines
6 — IA & Polish	Mapping IA, templates, notifications, webhooks, dark mode	4-6 semaines
7 — Tests & Launch	Tests E2E, sécurité, performance, beta privée, lancement public	3-4 semaines

Durée totale estimée : 24 à 35 semaines (6 à 9 mois), selon la vélocité de l'équipe.

12. Évolutions post-MVP

Les fonctionnalités suivantes sont identifiées pour les versions ultérieures :

- API REST publique pour permettre aux développeurs tiers de lancer des migrations programmatiquement
- Connecteurs supplémentaires : API REST tierces, Google Sheets, FTP/SFTP
- Templates partageables entre membres d'une organisation
- Monitoring avancé : Loki/Grafana, alertes Slack/Discord, métriques de performance
- Migrations incrémentales / planifiées (synchronisation récurrente)
- Marketplace de templates communautaires (mappings préconfigurés pour des couples logiciels populaires)
- Application mobile (consultation du dashboard, notifications push)
- Intégration directe avec des logiciels métier via leurs API (ex : Sage, Salesforce, HubSpot)
- Mode on-premise / self-hosted pour les grandes entreprises
- Support multi-région (hébergement géographiquement distribué)

13. Glossaire

Terme	Définition
MVP	Minimum Viable Product — première version fonctionnelle du produit
Mapping	Correspondance entre les champs du schéma source et du schéma cible
Dry-run / Preview	Exécution simulée d'une migration pour vérifier le résultat avant consommation de crédits
Tenant	Entité logique isolée (organisation) dans une architecture multi-tenant
FK (Foreign Key)	Clé étrangère — référence entre deux tables liées
RabbitMQ	Broker de messages utilisé pour le traitement asynchrone des migrations
MinIO	Serveur de stockage objet compatible avec l'API Amazon S3
Paddle	Plateforme de paiement agissant comme Merchant of Record
Dead Letter Queue	File d'attente recevant les messages échoués pour analyse
Webhook	Appel HTTP automatisé vers une URL configurée lors d'un événement
SSR	Server-Side Rendering — rendu côté serveur (Next.js)
AES-256	Standard de chiffrement symétrique à 256 bits